

Profiling protokol – výpočet smerodajnej odchýlky

1. Použitý nástroj

Na profilovanie programu bol použitý nástroj cProfile

```
python3 -m cProfile -s tottime stddev 1 2 3 4 5 6 7 8 9 10
```

2. Výstup profileru

3.0276503540974917

240 function calls in 0.001 seconds

Ordered by: internal time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	0.000	0.000	stddev:42(compute_stddev)
11	0.000	0.000	0.000	0.000	stddev:19(read_numbers)
1	0.000	0.000	0.000	0.000	stddev:66(main)
1	0.000	0.000	0.000	0.000	stddev:5(sqrt)

3. Analýza výsledkov

Celkový čas vykonania programu je približne 0.001 sekundy, čo znamená, že program je veľmi rýchly. Najdôležitejšie časti programu sú `read_numbers()`, `compute_stddev()` a `sqrt()`. Väčšina ostatných volaní pochádza z interných operácií Pythonu

4. Úzke miesta

Najviac času sa relatívne spotrebuje pri načítaní vstupu (`read_numbers`) a samotnom výpočte (`compute_stddev`). Výpočet odmocniny má zanedbateľný vplyv

5. Možnosti optimalizácie

Algoritmus je už efektívny ($O(n)$). Optimalizácia by sa mala zamerať najmä na vstupné operácie, ako je konverzia `float()`. Pre veľké datasety by bolo vhodné použiť knižnicu NumPy alebo numericky stabilnejší algoritmus

6. Záver

Program je efektívny a vhodný pre daný účel. Najväčší čas je spotrebovaný pri spracovaní vstupu, nie pri samotnom výpočte.